Swarm Field literature review

Amartya Natayu, Abhronil Chaudhuri,

Tycho Grapendaal, Jin Kuo, Błażej Łytkowski, Izzy van der Giessen

September 2026

1 Field, crop and platform model

1.1 Introduction

Five EU cultivation setups, described only in terms of what the simulation must represent. Section 1.3.1 defines the parameter list; each scenario in Section 1.4 gives values for those parameters and nothing outside the list, with Table 1 holding the full set of values. Radio, sensing, energy and platform parameters belong to Step 2. All lengths are in metres, all times in days; field extent is given as $x \times y$ dimensions rather than area, since that is what the simulation instantiates.

1.2 Terminology

Terms used repeatedly below, with their effect on a robot rather than their full definition.

Odometry
Estimating position by counting wheel rotations and integrating over time, with no external reference. Also called dead reckoning. Error accumulates with distance travelled, so a robot running on odometry alone drifts further from its true position the longer it goes without a fix.

Wheel slip
The wheel turning further than the robot actually moves, because the tyre shears against loose or wet soil. It corrupts odometry directly: the robot believes it has travelled further than it has. Slip varies with soil moisture, so the error is not a fixed bias that can be calibrated out.

GNSS / RTK
Satellite positioning, and the correction technique that improves it from metres to centimetres using a fixed base station. RTK is what makes per-plant action feasible, but it needs a clear view of the sky and a live correction link, both of which degrade under a canopy.

Canopy closure
The point in the season when neighbouring rows grow together and the lane between them stops being passable. Before closure a robot can drive between rows; after it, it cannot, so the same field changes topology partway through a run.

Canopy radio loss
The extra attenuation a radio link suffers when transmitter and receiver sit low down inside vegetation rather than in the open. Stems and leaves absorb and scatter the signal and block the direct path, so the usable range between two robots falls by roughly a factor of three to four once the crop is tall. It is why robots may need to forward messages for each other rather than all talking to one base.

Tramline
A permanent unplanted wheel track left through a dense crop so machinery can enter a field that has no usable inter-row lanes. Movement is confined to these tracks.

Headland
The unplanted strip at the ends of the rows, used for turning. A robot cannot turn mid-row, so every direction change costs a trip to the headland.

Heavy-tailed dispersal
A spread pattern where most offspring land nearby but a small fraction travel very far. It means a new outbreak can appear anywhere in the field rather than only next to an existing one, so searching only around known targets is not sufficient.

Flush / wave
In mushroom cultivation, a synchronised batch of fruiting bodies maturing together. Production arrives in waves rather than continuously, so demand for harvesting is strongly peaked.

1.3 Common model

All five scenarios reduce to cells with a connectivity graph; no continuous-space agent motion is required. Infestation density, slope and end-effector accuracy are attached to cells as values (scalar, edge cost, and per-action success probability) rather than simulated as dynamics.

1.3.1 Parameter list

Name	Meaning
Field geometry	
`field_x`, `field_y`	Field extent.
`cell`	Side length of one square cell. Chosen so a cell holds at most one target.
`row_sp`	Row spacing. Aisle pitch in S2 and S5.
`in_row_sp`	Plant spacing along a row. `0` = continuous stand, no addressable individual plants.
`headland`	Turning strip at row ends. Direction changes happen only here.
`slope`	Ground slope. Uphill and downhill edge costs differ.
`layers`	Stacked cultivation levels. `1` for all field scenarios.
Crop state	
`canopy`	Crop height at maturity. Drives occlusion; input to the Step-2 sensing and radio models.
`closure_day`	Day the canopy closes and inter-row lanes stop being passable. `---` = never.
`season_len`	Length of one production cycle.
`crops`	Distinct crop types present simultaneously.
Traversability	
`max_body`	Largest body fitting between rows; `row_sp` minus clearance.
`lane_access`	Do lanes stay usable after `closure_day`?
Targets and actions	
`targets`	Applicable presets, P1–P4 (Section 1.3.2).
`granularity`	Smallest addressable unit: `plant`, `row`, `zone`.
`reversible`	Whether a wrong action can be undone. Sets the false-positive cost.

	S1 Open field			**S2**	**S3**	**S4**	**S5**
	beet	wheat	maize	greenhouse	strip/pixel	vineyard	mushroom
`field_x` [m]	100–700	100–700	100–700	100–300	30–700	100–400	15–40
`field_y` [m]	100–700	100–700	100–700	100–300	30–700	100–400	10–25
`cell` [m]	0.25	0.5	0.75	0.45	0.5	1.0	0.1
`row_sp` [m]	0.45–0.50	0.125–0.20	0.70–0.80	1.6	0.5–24	1.8–2.2	1.48
`in_row_sp` [m]	0.15–0.25	0	0.13–0.20	0.44–0.55	per crop	0.9–1.2	0.1
`headland` [m]	3–12	3–12	3–12	rail ends	3–12	3–6	rack ends
`slope` [%]	0–5	0–5	0–5	0	0–5	**0–20**	0
`layers` [–]	1	1	1	1 †	1	1	**5–6**
`canopy` [m]	0.4–0.6	0.7–1.0	2.5–3.0	3.2	0.3–2.0	1.5–2.0	—
`closure_day` [d]	56–70	20–30	42–56	—	per crop	—	—
`season_len` [d]	180–210	250–300	140–170	300–330	60–300	365	21–30
`crops` [–]	1	1	1	1	**6**	1	1
`max_body` [m]	0.35	—	0.60	0.60	by `row_sp`	0.80	0.60
`lane_access` [–]	yes	**no**	yes	always	permanent	always	always
`targets` [–]		P1, P2, P3		P1	P1–P3‡	P3	**P4**
`granularity` [–]	plant	**zone**	plant	plant	plant	plant	plant
`reversible` [–]	yes	yes	yes	yes	yes	**no**	n/a

Table 1: Scenario configuration. †One ground layer, but three vertical sensing bands per cell. ‡With the `host_match` modifier active.

1.3.2 Target-dynamics presets

Distinct pest species do not need distinct models. What the simulation sees is a spawn rule, a growth rate, a spread rule and a detection window. Grouped that way, four presets cover all five scenarios. Aphids, whitefly and *Tuta absoluta* all collapse into P1: they arrive anywhere, multiply fast and disperse far, which is all the simulation needs.

	P1 Mobile pest	P2 Static patch	P3 Radial focus	P4 Harvest target
Example	aphid, whitefly, *Tuta*	weed	fungal disease	mushroom, ripe fruit
Spawn	point foci plus random arrivals anywhere	point, at field edges and entrances	point foci, or edge-in	regenerates in waves across the whole bed
Growth	exponential, doubling 2–3 d	slow, roughly linear	bursty, gated on weather events	deterministic ripening
Spread	long-range, heavy-tailed: ∼50 % of dispersal beyond 90 m	under 0.5 m per season, elongated along traffic	radial, metres per week	none
Detection window	days	weeks, early season only	days to weeks	**hours**
On action	target removed	target removed	target removed	target collected; removal *accelerates* the next wave

Table 2: Target presets. P1–P3 are threats to remove; P4 is a good to collect, and is the only preset where acting changes future target generation.

One modifier applies on top of a preset: `host_match`, used only in S3, multiplies spread probability by a factor between 0 and 1 depending on whether the destination cell holds a host crop. With `crops` = 1 it is always 1 and has no effect.

1.4 Scenarios

1.4.1 S1 — Open-field row crops

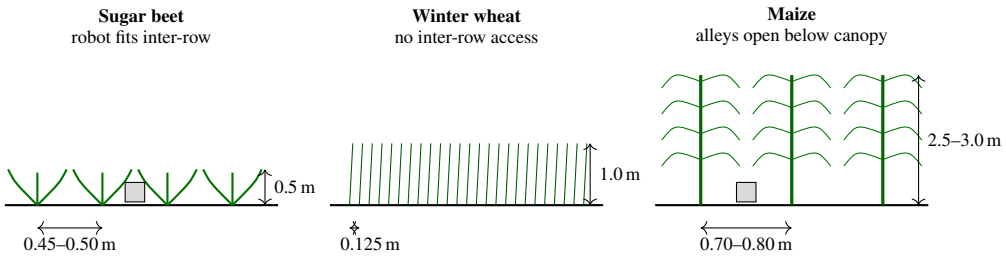


Figure 1: S1 cross-sections at maturity, to scale within each panel. Grey block is a 0.35 m robot body.

Three presets of one model: same grid engine, different numbers (Table 1).

Notes.

- Wheat is the case that breaks inter-row driving. At 0.125 m spacing no body fits, so `granularity` drops to `zone` and movement is confined to pre-planned tramlines.
- `canopy` rises through the season, so `closure_day` switches `lane_access` off partway through a run: a state change inside one simulation, not a separate scenario.
- Jobs: SCOUT, VERIFY, REMOVE, RELAY.

1.4.2 S2 — Greenhouse tomato

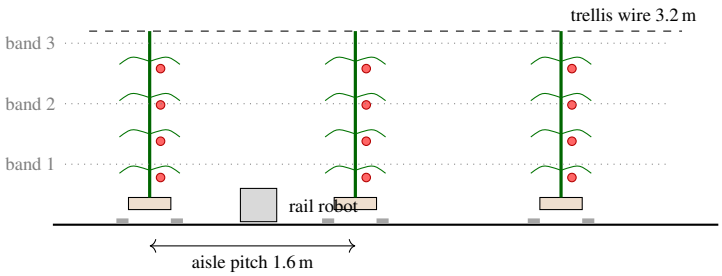


Figure 2: S2 cross-section. Floor heating pipes act as rails. Targets are spread along a 3 m vertical stem, discretised into three sensing bands.

Notes.

- **Cells sit on a rail graph, not a grid.** A robot moves along a rail and can change rail only at the ends. This is the one scenario where connectivity differs structurally from the others.
- **Targets are vertically distributed.** A sensing action observes one of three bands, not the whole cell. Choosing which band has no analogue in the flat scenarios.
- **Spawn is edge-in:** P1 targets enter at vents and doors, so boundary cells have a higher arrival rate.
- **DISPERSE is available here and nowhere else.** Control is biological: predators deposited at a chosen cell spread from that point over days, so the allocation problem is choosing release cells against a *predicted* distribution rather than acting on confirmed targets.
- Rail motion removes wheel slip, odometry drift and canopy radio loss. Whether that makes S2 a useful baseline for isolating coordination performance, or just an easier scenario, is for the team to decide. Jobs: SCOUT, VERIFY, DISPERSE, COLLECT.

1.4.3 S3 — Strip and pixel cropping

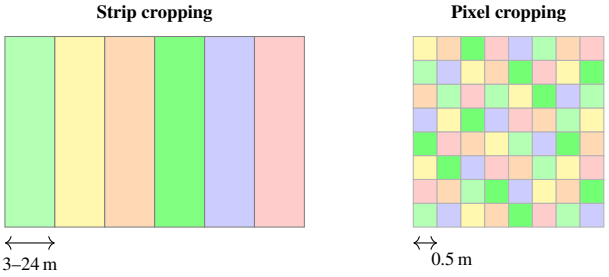


Figure 3: S3 plan views. `row_sp` spans 0.5 m (pixel) to 24 m (commercial strip) as a single parameter; `crops` = 6 in both.

Notes.

- **The only scenario with `crops` > 1**, which activates `host_match`: a target spreading into a non-host cell has its establishment probability multiplied down towards zero. `row_sp` therefore becomes a control variable rather than fixed geometry. Field trials report lower disease spread at 3 m and 6 m strip widths than in a uniform reference.
- **`cell` matches reality exactly.** Pixel-cropping trials use a literal 0.5 m × 0.5 m grid, so the simulation grid is not an approximation here.
- **Actions differ between adjacent cells.** Neighbours can hold different crops at different stages needing different actions on different days, so units must either carry every tool or specialise by tool and be routed accordingly — specialisation on two axes: sense-versus-act and tool type.
- Strip width is currently bounded below by machine working width; at one on-farm trial the pixel size was chosen to fit the farmer's existing machinery. Jobs: all six.

1.4.4 S4 — Vineyard

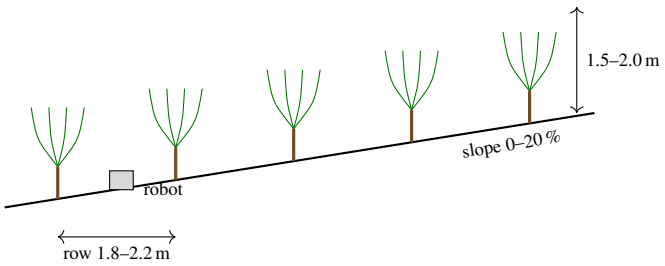


Figure 4: S4 cross-section. Vines are permanent obstacles at fixed, known positions.

Notes.

- **`reversible` = no.** Removing an infected vine destroys a perennial plant. The only scenario where a false positive is permanently costly, so the only one forcing an explicit confidence threshold and mandatory VERIFY before REMOVE.
- **`slope` is non-zero.** Uphill and downhill edge costs differ. Every other scenario can run with `slope` = 0.
- **Plants are permanent and their positions known.** The plant map persists across seasons, so per-cell records accumulate over years. Jobs: SCOUT, VERIFY, REMOVE.

1.4.5 S5 — Indoor mushroom

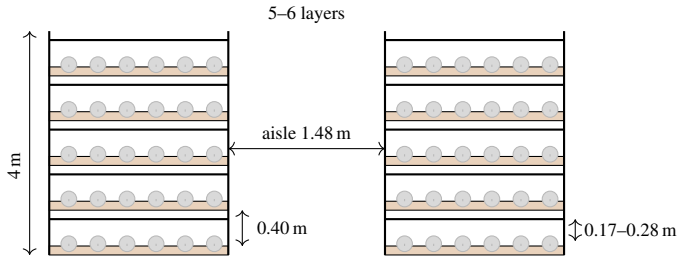


Figure 5: S5 cross-section. Cap-to-shelf clearance of 0.17–0.28 m bounds any manipulator.

Notes.

- **The only P4 scenario, and the only one with `layers` $>$ 1.** Targets are goods to collect, not threats to remove, and cells stack with connections only at rack ends.
- **Acting changes future target generation.** Harvesting removes a growth inhibitor, so it accelerates the next wave. Every other preset leaves future spawns unaffected by the robot, making this the only non-greedy scheduling problem in the set.
- **The detection window is hours.** Picking early loses yield; picking late costs quality and delays the next wave.
- Waves arrive on a 7–10 day cycle, each harvested over 2–4 days with two or three passes per day; density and overlap fall with each successive wave.
- Structurally P4 is P1 with the sign flipped: distributed targets appearing over time, each with a window, scheduled under a resource constraint. Jobs: SCOUT, COLLECT.

1.5 Simulation requirements

1. **Cell and graph engine.** Configurable `cell`, in three modes: 4/8-neighbour grid with per-edge cost (S1, S3, S4); rail graph with connections only at row ends (S2); vertical stacking with connections only at rack ends (S5). One implementation covers all five.
2. **Crop state layer.** Per-cell crop type, growth stage and `canopy`; `canopy` must be readable by the Step-2 sensing and radio models.
3. **Target generator.** Implements presets P1–P4, plus the `host_match` multiplier for S3 and the harvest-feedback coupling for P4.
4. **Detection model.** Returns a probability, not a binary result, depending on sensor footprint and dwell time. S2 additionally needs per-band detection.
5. **Agent model.** Position, battery, tool inventory, failure probability; ground and aerial agents share one interface.
6. **Action model.** One interface across the six job types returning success, cost and side effect. REMOVE deletes a target; DISPERSE writes a decaying field around the release cell; COLLECT deletes a target and modifies the next spawn wave.
7. **Metrics.** Target count over time, detection latency, energy per unit area, messages sent, and degradation when k agents fail.

Scope. Building all five may exceed available effort. S1 and S3 between them exercise the widest range of the parameter list; S2, S4 and S5 each add exactly one structural feature (rail graph, irreversible actions, stacked layers with harvest feedback) and could be added incrementally.

2 Robot hardware

2.1 Introduction

This section covers what a swarm unit has to carry, what that costs in power, mass and money, and which of those constraints actually bind. The aim is not to select hardware but to bound it: for each subsystem we give two or three alternatives with their trade-offs, and a parameter range the simulation can sample from.

Three quantities are tracked for every component: **power draw** (sets endurance), **mass** (sets whether a flying platform can carry it at all) and **cost** (sets how many units the swarm can have, which is the premise of the whole project). Cost figures are component-level estimates at small volume, not quotes, and should be treated as order-of-magnitude.

Section 2.6 is the part worth reading first if time is short: it separates the constraints that genuinely limit the design from the ones that look restrictive but are not.

2.2 Terminology

Shorthands used below. Terms already defined in the field and crop section (odometry, wheel slip, GNSS/RTK, canopy radio loss) are not repeated.

W, Wh
Watt is power, watt-hour is energy: a 10 W load for 2 h consumes 20 Wh. Endurance is battery capacity divided by average draw.

Specific energy (Wh/kg)
How much energy a battery stores per kilogram. It decides how much endurance a given mass budget buys, and it is the single number that makes flying hard.

IMU
Inertial measurement unit: accelerometers plus gyroscopes on one chip. Gives orientation and acceleration at high rate, but its position estimate drifts, so it is always used alongside something that provides absolute correction.

UWB
Ultra-wideband. A short-range radio used to measure the *distance* between two units by timing a pulse between them. It returns a range, not a position; positions come from combining several ranges.

ToF
Time of flight. Measuring distance by timing a pulse, used by both UWB (radio) and depth cameras (infrared).

TDMA
Time-division multiple access. Units take turns transmitting in scheduled slots so they do not talk over each other. The cost is that the more units share a channel, the less often each one gets to speak.

Duty cycle
The fraction of time a radio is legally allowed to transmit. European sub-GHz bands cap this at 1 %, which reduces usable throughput by a factor of 100 regardless of the radio's raw bit rate.

SBC
Single-board computer, e.g. Raspberry Pi or NVIDIA Jetson. The robot's main processor, as distinct from the microcontroller that drives the motors.

TOPS
Tera-operations per second. A throughput figure for AI accelerators. Useful only for rough comparison, since achieved performance depends heavily on the model and on quantisation.

INT8 quantisation
Running a neural network with 8-bit integers instead of 32-bit floats. Roughly quadruples throughput and cuts power for a small accuracy loss. It is why a 7 W board can run real-time object detection.

ESC
Electronic speed controller. The driver between the battery and a brushless motor. One per motor, so a quadcopter carries four.

VIO
Visual-inertial odometry. Estimating motion by tracking image features and fusing with the IMU. A drift-correcting alternative to wheel odometry that needs no wheels, so it works in the air.

IP rating
Ingress protection, written as two digits: dust then water. IP65 means dust-tight and resistant to water jets; IP67 adds temporary immersion. Field robots need at least IP65.

MTBI
Mean time between interventions. How long a robot runs before a human has to touch it. More useful than MTBF for fielded systems, because most stoppages are not component failures.

Payload fraction
Useful mass divided by total mass. For multirotors it is typically 10–30 %, which is what makes every gram of sensor and

compute expensive.

Rolling resistance coefficient (C_{rr})

The fraction of a vehicle's weight that must be overcome as drag when rolling. About 0.01 on tarmac, 0.1–0.3 on soil, which is why ground robots spend far more energy per metre in a field than on a road.

2.3 Component inventory

Component	Power [W]	Mass [g]	Cost [€]	Note
Microcontroller (motor control)	0.1–0.5	10	5–20	always on
SBC, Pi-class	5–10	50	60–120	no AI accelerator
SBC, Jetson-class	7–25	150	250–450	7 W eco mode usually sufficient
RGB camera	1–2	30–80	30–200	
Depth / ToF camera	1.2–3.5	60–100	150–400	swappable
Multispectral camera	2–4	100–200	800–3000	swappable, dominates sensor cost
2D LiDAR	2–5	150–200	100–400	swappable
IMU	<0.1	5	10–50	
GNSS receiver	0.1–0.5	20	20–60	metre-level
RTK GNSS receiver	0.5–1.0	30–50	200–800	plus one shared base station
UWB module	0.1–0.3	5–10	20–50	duty-cycled
Sub-GHz radio (LoRa-class)	<0.01 avg	5–10	10–30	1 % duty cycle
Wi-Fi / BLE module	0.5–2	5–10	5–20	short range, high rate
Li-ion cells	—	5–7 g/Wh	0.3–0.6 /Wh	150–250 Wh/kg
$LiFePO_4$ cells	—	8–11 g/Wh	0.3–0.5 /Wh	90–120 Wh/kg, longer cycle life
Ground drive motor + gearbox	20–60 ea	300–800	40–150	2–4 per robot
Brushless motor + ESC (air)	40–80 ea	60–120	30–80	4 per quadrotor
Solar panel	generates	1.2–2 kg/m^2	100–300 /m^2	15–20 % efficient

Table 3: Indicative component figures at small volume. Power is typical draw in use, not peak.

Two observations from the table that shape everything downstream. First, **the multispectral camera and the RTK receiver dominate per-unit cost** and are the two components most worth making swappable or shared rather than fitting to every robot. Second, **sensing and compute together draw 10–30 W while drive motors draw 40–250 W**, so locomotion sets endurance and everything else is a rounding error — except on a flying platform, where the 15 W of avionics also has to be lifted.

2.4 Subsystems

2.4.1 Movement

Option	Fits	Speed [m/s]	Drive power [W]	Trade-off
Differential wheels	all ground scenarios	0.3–1.3	40–80 (25 kg)	default; simple, cheap, turns in place
Tracks	wet or soft soil	0.2–0.8	60–120	lower ground pressure, higher energy and steering loss
Rail carriage	S2 only	0.3–1.0	20–40	no slip or drift at all; cannot leave the rail
Multirotor	any, from above	5–12	150–250	20–45 min endurance; cannot stop cheaply
Legs	—	—	—	rejected: no fielded agricultural precedent, buys clearance not needed on prepared ground

Ground energy. Drive power follows $P = C_{rr}\, m\, g\, v/\eta$. For a 25 kg robot at 0.5 m/s with $C_{rr} = 0.2$ and drivetrain efficiency $\eta \approx 0.5$ this gives about 50 W, or **100 J per metre travelled**. Two independent checks agree: a 150 kg commercial weeding robot carries 4×110 W rated motors, which scales to ∼70 W rated at 25 kg, and a fielded 27 kg solar mower runs all day on a panel that cannot exceed ∼50 W peak.

Air energy. Hover power is

$$P_{\text{hover}} = \sqrt{\frac{(mg)^3}{2\pi r^2 \rho}},$$

with m the total mass, r the rotor radius and ρ air density. A 5 kg platform with 0.5 m rotors needs about 247 W; a 1.5 kg platform with 0.12 m rotors about 170 W. The structural point is that $P \propto m^{1.5}$: adding battery to extend endurance adds mass, which raises power super-linearly, so returns diminish quickly. Doubling the battery on a 1.5 kg airframe buys perhaps 60 % more flight time, not 100 %.

The asymmetry that matters. Measured energy per metre is comparable between the two — roughly 100 J/m on the ground and 80 J/m for a small quadcopter carrying 0.5 kg. The difference is what happens when the robot *stops*: a parked ground robot draws about 15 W, a hovering drone draws nearly full power. For a swarm that spends most of its time observing rather than transiting, this is decisive, and it is the reason the two platform types are complements rather than alternatives.

On modelling acceleration: do not. At 0.5 m/s with 0.25–1 m cells a cell takes 0.5–2 s to cross, while acceleration to full speed takes well under a second, so acceleration only perturbs sub-cell timing. A drone crosses cells in 0.1 s but flies straight over many in sequence, so acceleration amortises. What does matter, and costs two constants instead of an integrator, is a fixed `turn_cost` (headland turns are unavoidable and slow) and a fixed `stop_cost` per action.

2.4.2 Positioning

The question is not whether the swarm needs GNSS but whether it needs a *shared coordinate frame*, which is separable. Navigation needs relative position; map fusion between robots needs a common frame.

Option	Accuracy	Cost/unit [€]	Scaling	Fails when
RTK GNSS	8–30 mm	200–800	$O(1)$	sky blocked, or correction link lost
GNSS alone	0.5–2 m	20–60	$O(1)$	sky blocked
UWB inter-robot	centimetre	20–50	$O(n^2)$	swarm grows past channel capacity
UWB + fixed anchors	centimetre	20–50 + anchors	$O(n)$	anchors must be deployed per field
Crop-row visual	row-relative	0 (reuses camera)	$O(1)$	no long rows (S3 pixel), canopy closed
Dock reset	resets drift	0	$O(1)$	— (periodic only)
Wheel odometry + IMU	drifts	10–50	$O(1)$	always available, always degrading

Relative positioning is viable. UWB gives centimetre-level relative localisation without infrastructure, at low cost, low power and high update rate, and anchor-free schemes exist that fuse UWB ranges with IMU and monocular vision alone. So a swarm can localise itself against itself.

But the scaling argument runs the other way from intuition. GNSS is receive-only and passive, so it is $O(1)$ per robot and scales perfectly to any swarm size. UWB two-way ranging is pairwise and scheduled by TDMA, so the number of ranging exchanges grows as $O(n^2)$ and each robot's update rate falls as the swarm grows. Five robots is comfortable; fifty needs clustering or neighbour subsetting. **Relative positioning is cheaper per unit and more robust; global positioning scales better.** This is a genuine trade-off and should be presented as one.

The robustness case is nonetheless strong. RTK already has two points of failure — the constellation and a live correction stream from a base station — and GNSS can be degraded by jamming or interference at the receiver, which is a more realistic event than a satellite failing. A swarm that degrades gracefully to row-following plus UWB ranging plus a dock reset on each recharge survives losing either. Since robots must return to charge anyway, **the dock reset is a free periodic global fix** and costs no hardware at all.

Recommendation for the simulation. Model `pos_mode` as a selectable enum with an accuracy and an availability, and make availability a scenario property rather than a robot property. The interesting experiment is whether swarm performance degrades gracefully as `pos_mode` is downgraded mid-run.

2.4.3 Sensing

The useful split is not sensors versus actuators but **what makes a unit a swarm member** versus **what makes it able to do a job**. This maps directly onto a detachable-head architecture.

Head — on every unit		**Body** — swappable by task	
IMU	orientation, motion	Depth / ToF camera	3D targets (S2, S5)
Wheel encoders, motor current	odometry, fault detection	Multispectral camera	vegetation discrimination
Sub-GHz radio	swarm membership	2D LiDAR	dense obstacles (S4)
Short-range radio	bulk sync on encounter	Macro camera	pest-scale detection
UWB	relative position	Effector	hoe, laser, sprayer, dispenser
RGB camera	navigation, basic detection	Consumable tank	spray, beneficials
Battery monitor	endurance, degradation		

Head cost lands around €400–700 and head power around 10–15 W. A body adds €150–3000 and 2–5 W depending on what it carries; the multispectral option alone can exceed the cost of the head. That asymmetry is the argument for bodies being swappable rather than every unit being fully equipped.

Detection is modelled as a probability rather than a range: a nadir camera has a *footprint* of 0.2–0.6 m at close range, and whether a target inside that footprint is seen depends on resolution, dwell time and occlusion. There is no meaningful "detection radius" for millimetre-scale targets.

2.4.4 Communication

Option	Rate	Range (in canopy)	Power	Role
Sub-GHz (LoRa-class)	0.3–50 kb/s	50–300 m	<0.01 W avg	event reporting, always on
Wi-Fi / BLE	1–50 Mb/s	10–50 m	0.5–2 W	bulk sync on encounter
Cellular (4G/5G)	Mb/s	site-dependent	1–3 W	needs coverage and subscription per unit
Aerial relay	as above	3–4× better	—	drone above canopy as mobile gateway

Two distinct requirements, often wrongly conflated. Mission data (observations, task claims, map updates) is latency-tolerant: with targets doubling on a 2–3 day timescale, a ten-minute message delay changes nothing. Collision avoidance and formation keeping are latency-critical at roughly 10 Hz but only between near neighbours. These should use different channels and be sized separately; only the second has a hard real-time requirement, and it is short-range by nature.

Data volume. A minimal observation record is about 10 bytes (cell identifier, state, confidence, timestamp), and roughly 25 bytes on the wire after protocol overhead. A ground robot at 0.5 m/s over 0.5 m cells generates one observation per second, so **10 B/s of raw state**. A sub-GHz radio at 5.5 kb/s with a 1 % duty cycle delivers about **7 B/s per device**.

Those numbers are uncomfortably close, but only if the robot transmits everything it observes. If it transmits only events — detections, claims, completions — and a typical field has targets in a few percent of cells, the offered load drops to well under 1 B/s. **Sub-GHz is sufficient for event reporting and insufficient for continuous state streaming**, and that distinction is an architectural requirement, not an optimisation.

Full map synchronisation is not possible over the long-range link, at all. A 700 m field at 0.5 m cells is about 2 million cells; at 8 bytes each that is 16 MB. Over a 5 kb/s link that is roughly **seven hours** to send once. Over a 10 Mb/s short-range link it is **13 seconds**. This single comparison justifies the whole two-tier architecture: gossip events continuously over sub-GHz, and reconcile maps opportunistically over short-range radio when two units happen to meet.

Channel access is the real limit, not bit rate. A 25-byte message at 5.5 kb/s occupies the channel for roughly 36 ms. Random-access channels saturate well below full utilisation, so a practical ceiling is around 5 messages per second on one channel. At one message every few seconds per robot, **one sub-GHz channel supports roughly 20 robots**. Beyond that the swarm needs multiple channels, multiple spreading factors, or spatial reuse — all of which exist, but they are design work rather than a free parameter.

Aerial units have a different problem. A drone at 10 m/s over 1 m cells generates observations 20× faster than a ground unit, so it must aggregate a survey swath and transmit a summary rather than streaming. In exchange it gets a far better link: with the transmitter clear of the canopy, measured range rises from roughly 700 m to over 2500 m. A drone at 20 m altitude is therefore a **3–4× range multiplier acting as a mobile gateway**, which is arguably the strongest argument for a mixed air/ground swarm.

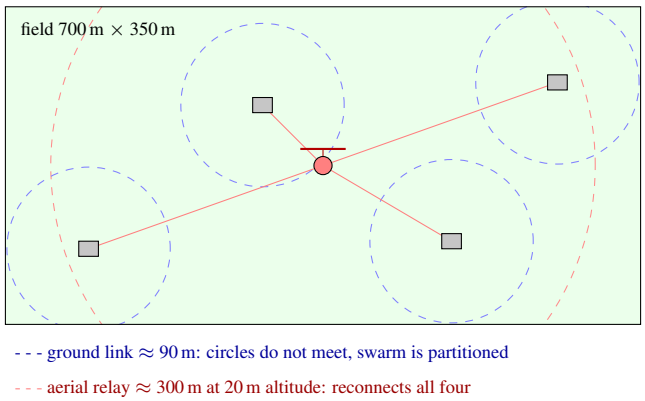


Figure 6: Range against field extent, drawn to scale. In-canopy ground links do not span the gaps between units, so the ground swarm is frequently partitioned; one unit above the canopy reconnects it.

2.4.5 Power

Budget. For the simulation, express energy as a standing draw plus a per-metre and a per-action cost, rather than integrating instantaneous power:

$$E = P_{\text{idle}}\, t + e_{\text{move}}\, d + e_{\text{act}}\, n_{\text{actions}}.$$

For a 25 kg ground unit: $P_{\text{idle}} \approx 15$ W, $e_{\text{move}} \approx 100$ J/m. Total draw is 45–85 W moving and 12–25 W parked but sensing. With 300–800 Wh aboard that gives 5–13 h, which matches fielded platforms quoting 8–10 h on lithium.

Battery chemistry. Li-ion at 150–250 Wh/kg maximises endurance per kilogram and is the only realistic choice for flight. $LiFePO_4$ at 90–120 Wh/kg is heavier but tolerates far more charge cycles and is safer to charge unattended in a field shed — relevant for a swarm that cycles daily for a season. For ground units the extra mass is nearly free, so the cycle life probably wins.

Recharge versus swap. Charging takes 1–4 h and removes the unit from the pool for that whole period. Swapping a pack takes minutes but requires either a human or a logistics robot (platform C below). **The choice changes the shape of the availability curve, not just its level**, and is worth exposing as a simulation parameter rather than assuming one.

Solar is not sufficient in the Netherlands, and this is worth stating with the number. Annual mean insolation is roughly 2.8 kWh/m^2/day. A 0.3 m^2 panel at 18 % efficiency yields about 150 Wh/day, which sustains **roughly 6 W continuous**. That covers standby, intermittent sensing and very slow motion — which is exactly the niche the fielded 27 kg solar mower occupies — but it is an order of magnitude short of the 45–85 W a working ground unit draws. Solar extends a duty cycle; it does not replace charging.

2.4.6 Computation and memory

Option	Power [W]	Cost [€]	Suitable for
Microcontroller only	0.1–0.5	5–20	motor control, sensor polling, radio; no vision
Pi-class SBC	5–10	60–120	navigation, row following, classical vision
Jetson-class SBC	7–25	250–450	neural detection, VIO; 7 W mode usually enough

Compute is over-provisioned at any sensible choice. A Jetson-class board in its 15 W mode runs small detection networks at hundreds of frames per second under INT8 quantisation, while the sensing speed limit of 0.45–0.89 m/s is set by image quality and actuator response, not by inference. Running the 7 W eco mode instead costs throughput the robot was never going to use.

Memory is not a constraint either, and the consequence is interesting. The 16 MB full-field belief map computed in Section 2.4.4 fits in any of these boards with three orders of magnitude to spare. **Every robot can hold a complete map of the field.** The limit is therefore entirely bandwidth to synchronise, never storage — so the simulation should model message cost carefully and memory cost not at all.

One coupling worth noting. Power mode selection ties directly to thermal design: 25 W requires active cooling, which requires a vent, which breaks the ingress seal. Passive cooling at 7 W in a sealed enclosure is feasible; 25 W in a sealed enclosure in direct sun is not. **Compute budget and IP rating are not independent choices.**

2.4.7 Mechanical robustness

IP65 is the outdoor minimum (dust-tight, resistant to water jets). Mushroom rooms are high-humidity washdown environments and want IP67. Operating range outdoors is roughly -10 to $+45\,^{\circ}\mathrm{C}$, with thermal throttling a real risk for a sealed box in sun. Vibration over ruts is the routine killer of connectors and mounts rather than of components themselves.

Agricultural robots in unstructured environments still produce inadequate results because topography, soil, vegetation, visibility, illumination and weather change on scales from seconds to months and millimetres to kilometres. For the simulation this argues for treating hardware capability as *degrading* rather than binary.

2.4.8 Outages

Partial failure is worse than complete failure. Swarms are not reliably tolerant of individual failures, and are least tolerant of units that have only partially failed and continue contributing to collective behaviour. A dead robot is merely absent; a half-dead one keeps injecting bad data. This inverts the intuition that redundancy handles failure automatically, and it is the main reason to model failure as capability degradation rather than agent deletion.

Taxonomy to implement. Five types are standard in the literature: complete failure of both motors, complete failure of one motor, complete sensor failure, partial motor failure (torque degradation) and partial sensor failure (reduced range or field of view). Motor failure is the most damaging to collective behaviour.

Two mechanisms that are cheap to simulate and consequential. Degradation's main early effect is on power consumption, which approximately doubles before physical symptoms appear — so **power draw is a free fault detector**, which is why motor current sensing belongs in the head. And failed units act as obstacles that others must route around, which increases time spent in their vicinity and therefore the chance of failing there too, so **failures cluster and cascade**.

Frequency. A well-engineered autonomous platform in long-term industrial inspection reported 16 interventions over 81 missions, a mean time between interventions of about 140 h excluding minor ones, and a longest clean run of 15 days. Field conditions are harsher, so **20–100 h MTBI** is a reasonable planning range.

Repair or salvage? For cheap units, replacement rather than field repair. The swarm-relevant question is *retrieval*: a failed unit sitting in a field is simultaneously a lost asset and an obstacle. Note also that a unit with undetected motor degradation eventually becomes unable to move and depletes its battery before reaching a charging point — so there is a window in which early detection keeps the unit retrievable under its own power, and missing that window converts a cheap recovery into an expensive one.

2.5 Example platforms

Three configurations spanning the design space. Costs are component sums at small volume and exclude assembly, software and spares.

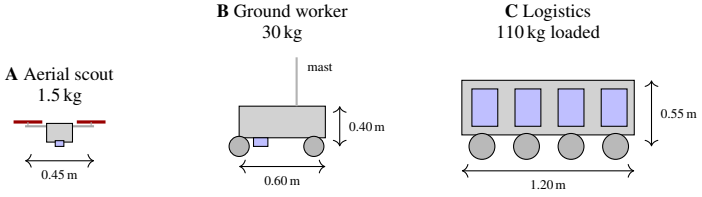


Figure 7: The three example platforms, drawn to a common scale. Blue blocks are battery packs.

2.5.1 A — Aerial scout

Mass budget		*Specification*	
Frame, 4 motors, 4 ESCs	500 g	Dimensions	0.45 m diagonal, 0.15 m tall
Battery (70 Wh Li-ion)	400 g	Cruise speed	5–12 m/s
Jetson-class SBC + carrier	150 g	Hover power	≈170 W
Enclosure, wiring	130 g	Avionics power	15–20 W
RGB camera	60 g	**Endurance**	**20–25 min**
GNSS, IMU, UWB, radios	60 g	Recharge	40–60 min
Payload margin	200 g	Cost	€1500–3500
Total	**1500 g**	Jobs	SCOUT, RELAY

Endurance is the binding constraint and is not improvable by much: at $P \propto m^{1.5}$, doubling the battery to 140 Wh raises total mass to about 1.9 kg and hover power to about 240 W, yielding roughly 35 min rather than 45. The 200 g payload margin is what permits a swappable body; a multispectral camera at 100–200 g fits, a LiDAR at 150–200 g fits only by removing something else.

Its second role is as a communication relay, and that may be worth more than its sensing: above the canopy it reconnects ground units that cannot otherwise reach each other (Figure 6).

2.5.2 B — Ground worker

Mass budget		*Specification*	
Chassis, wheels, 4 motors	12 kg	Dimensions	0.60 × 0.32 × 0.40 m
Enclosure, suspension	5 kg	Antenna mast	0.6–1.0 m
Battery (500 Wh)	3 kg	Speed	0.3–0.8 m/s
Head (sensors, SBC, radios)	1.5 kg	Drive power	40–60 W
Body module (tool + tank)	3–8 kg	Total draw	65–85 W moving, 15 W parked
		Endurance	**6–8 h**
		Recharge / swap	2–3 h / 3–5 min
Total	**25–30 kg**	Cost	€3000–8000

Width is set by the narrowest crop it must enter: 0.32 m clears the 0.45–0.50 m sugar beet rows with margin, fits maize and vineyard comfortably, and cannot enter wheat at all — which is correct, since nothing can.

One honest limitation to carry forward: traction scales with mass, and the usual rule is that a platform needs roughly 2.5× the draft force it must generate in weight. At 30 kg this unit can survey, spot-treat and harvest, but it cannot perform mechanical soil work. Either the swarm is sensing-and-spot-treatment only, or it is heterogeneous with a few heavier units.

2.5.3 C — Logistics and charging

Mass budget		*Specification*	
Chassis, wheels, 4 motors	45 kg	Dimensions	1.20 × 0.70 × 0.55 m
Own battery (2 kWh)	12 kg	Speed	0.5–1.2 m/s
Cargo: 8 × 500 Wh packs	24 kg	Drive power	120–180 W
Enclosure, swap mechanism	20 kg	**Endurance**	**11–16 h**
Head + sensors	2 kg	Swap time	3–5 min per unit
Spare margin	7 kg	Cost	€6000–15 000
Total	**110 kg**	Jobs	SERVICE

It does not fit between narrow rows, and that is a feature. At 0.70 m wide it can enter maize and vineyard but not sugar beet, so in narrow crops it must remain on headlands and permanent lanes while workers come to it. That asymmetry — **the charger is reachable rather than mobile** — is a real constraint that changes the allocation problem, since a worker's decision to recharge now costs a traverse to the field edge.

Why it is worth including at all. Eight packs recharge eight workers without any unit spending 2–3 h offline. With a 25-unit swarm at 6–8 h endurance, one logistics unit can keep the fleet near-continuously available, whereas static charging removes a third of the swarm at any time. The simulation should be able to compare both regimes; it is a cheap experiment with a large effect on effective fleet size.

Economics, for context. Twenty ground workers plus two logistics units plus four aerial scouts sums to roughly €100 000–200 000 at these component estimates. A single commercial straddling vineyard robot is priced around €175 000. The swarm is therefore in the same cost bracket as the machine it would replace, which is the premise worth testing rather than asserting.

2.6 What is and is not a bottleneck

Subsystem	Binding?	Limit
Compute	**No**	Detection needs ∼1–30 FPS; a 7–15 W board delivers hundreds
Memory	**No**	Full-field belief map ≈16 MB against 4–8 GB available
Radio bit rate (events)	**No**	<1 B/s offered against ≈7 B/s duty-cycled capacity
Radio bit rate (full map)	**Yes**	16 MB is ≈7 h on sub-GHz; must use short-range sync
Radio channel access	**Yes**	≈20 robots per sub-GHz channel before collisions dominate
Radio range	**Yes**	50–300 m in canopy against fields of hundreds of metres
Positioning accuracy	**No**	Row-following plus dock reset suffices for navigation
Positioning scalability	**Yes**	UWB ranging is $O(n^2)$; degrades past ∼20–30 units
Ground endurance	**No**	6–13 h covers a working day
Aerial endurance	**Yes**	20–45 min, and $P \propto m^{1.5}$ resists improvement
Solar self-sufficiency	**Yes**	≈6 W continuous in NL against 45–85 W demand
Ground mass	**No**	But traction needs mass: a 30 kg unit cannot do soil work
Aerial mass	**Yes**	10–30 % payload fraction; every sensor competes with battery
Reliability	**Partly**	20–100 h MTBI; partial failures more damaging than total ones

Table 4: Constraints that genuinely limit the design, and those that only appear to.

Three of these deserve emphasis because they shape the architecture rather than merely constraining it.

1. **Range, not bandwidth, is the communication problem.** The swarm generates very little data; it simply cannot reach far enough to deliver it. This makes store-and-forward and aerial relay first-class design elements rather than fallbacks.
2. **Aerial units are constrained by time, ground units by space.** A drone can reach anywhere but only for 20 minutes; a ground unit can work all day but cannot cross a closed canopy or a wet headland quickly. The complementarity is structural, not incidental.
3. **Nothing about compute or memory limits a swarm's size or intelligence.** Every unit can hold the full map and run real-time detection for under 15 W. What limits the swarm is getting information between units and keeping them powered — which is exactly where a decentralised design has something to prove.

2.7 Simulation parameters

Agent-level parameters contributed by this section, to sit alongside the environment parameters defined earlier.

Name	Unit	Meaning	**A** aerial	**B** ground	**C** logistics
`mass`	kg	Total mass including payload	1.5	25–30	110
`body_w` × `body_l`	m	Footprint; gates which rows it can enter	0.45 × 0.45	0.32 × 0.60	0.70 × 1.20
`v_max`	m/s	Travel speed	5–12	0.3–0.8	0.5–1.2
`v_sense`	m/s	Speed at which detection stays reliable	2–5	0.45–0.89	—
`e_move`	J/m	Energy per metre travelled	80–150	100	200–300
`p_idle`	W	Standing draw (hover, for aerial)	170–250	15	20
`p_sense`	W	Added draw while sensing	3–6	3–6	—
`p_compute`	W	Added draw for perception	7–15	7–15	5
`turn_cost`	s	Fixed penalty per direction change	1–2	5–15	10–20
`stop_cost`	s	Fixed penalty per action stop	1–2	2–4	—
`battery`	Wh	Usable capacity	70	500	2000
`recharge`	h	Time offline to recharge	0.7–1.0	2–3	4–6
`swap`	min	Time offline if pack is swapped	—	3–5	—
`radio_range`	m	Reliable link distance, canopy-dependent	500–2500	50–300	50–300
`radio_rate`	b/s	Effective rate after duty cycle	55	55	55
`msg_bytes`	B	Size of one observation message	25	25	25
`pos_mode`	—	rtk / gnss / uwb / row / odometry	gnss, uwb	row, uwb, dock	row, dock
`pos_accuracy`	m	Position error for the selected mode	0.5–2	0.01–2	0.01–2
`mtbi`	h	Mean time between interventions	20–100	20–100	20–100
`cost`	€	Unit cost, for fleet sizing	1500–3500	3000–8000	6000–15000

Table 5: Agent parameters and values for the three example platforms.

3 Exploration, Coverage & Navigation

Exploration generally assumes the pose of the robot is known or can be identified and instead abstracts focus to guiding the robot through the whole environment. Active SLAM and other integrated approaches, are some combinations of these [**?**].

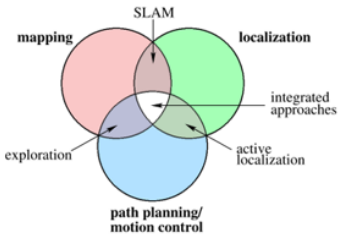


Figure 8: Venn digraph of the task a robot needs to solve in order to accurately create a model of an environment. Exploration can be seen in the intersection of path planning and mapping [**?**].

3.1 Exploration

Exploration is a decision making problem and therefore can most generally be formulated as a Partially observable Markov decision processes (POMDP). Due to the inherent computational intractability of POMDPs, most approaches reformulate this problem using an Information theoretic, Greedy, or Learning-based model [**?**].

If you want to know more about POMDP, please read the following citation [**?**].

3.1.1 Greedy-based

Move to the closest reachable position, with the highest utility, that is on the frontier or outside of what is know but presumably, in a safe and reachable location. The primary implementation of these methods are frontier and sampling based (doesn't mean the sampling based only belongs to greedy-based), as well as other myopic approaches.

Kuipers and Byun has developed an approach based on semantic hierarchy of spatial representation upon frontier-based method. The approach will be able to minimize vulnerability to their weaknesses: cumulative location error, uncertainty on navigation ability due to sensor and motor, navigation doesn't depend on geometric accuracy, geometric sensor fusion method can be incorporated and indistinguishable place [**?**].

There is also method that alters frontiers-based exploration to explicit preform loop closer that creates better maps by essentially ensuring frontiers are revisited [**?**].

An article has made a comparison between different approaches. It concludes that the closest frontier + repetitive re-checking has the shortest travel path for an open space, segmentation improves the performance in an office-like environment [**?**].

Matthew [**?**] has utilize four autonomous surface vehicles work together to explore water reservoirs in order to detect methane "hot-spots". They select sample locations using a random walk and potential fields. All robots communicate and represent previously sample sites as a 2D Gaussian; a random position at a set radius from each robots current position is selected and chosen to be sampled if not on land, and the value of the the closest Gaussian potential is below a threshold. The set radius changes based on sampling measures. Their method allows the robots to individually explore locations efficiently, while not overlapping with other robots [**?**].

Low et al. [**?**] presents an adaptive cluster sampling exploration approach that reduces mission time and energy consumption compared to other sampling approaches. Similar to [**?**] they are interested in detecting "hotspots", or locations of interest, rather than complete coverage.

A typical formulation of greedy-based exploration will be:

$$\arg\max_{u} \mathbb{E}\left[U(b|u)\right] - C(b) \tag{1}$$

3.1.2 Information gain-based

The information gain-based exploration is driven by a particular form of information and is non-myopic.

Atanasov et al. [**?**] recent work on information acquisition presents a truly non-myopic algorithm with performance guarantees and theoretical error bounds. Their focus is to optimize the trajectory of the robot, as it explores, in order to create more accurate maps. The problem can be reduced from a stochastic optimal control problem to deterministic

optimal control which reduces the state space of the sensors and allows offline computations. A forward value iteration approach, a search 'tree of trajectories' is created to represent the expected reachable states given a set of controls.

Exploration based on the expected next-best view is prove to be effective in indoor environment [**?**].

3.1.3 Learning-based

Learning-based is primarily data driven; the idea being that as robots explore a space, a policy of how to efficiently explore can be learned and used to explore similar environments. It is unclear if a learning based approach would be able to draw insights in unstructured environments such as the outdoors.

3.2 Coverage Path Planning

The artificial potential field (APF) algorithm is commonly used in detecting obstacles when the robot is towards the goal position. A fictional repulsive force and attractive force are created in the surrounding obstacles and around the goal respectively to ensure the robot in achieving the target while keeping the distance between the robot and obstacles [**?**].

Sutantyo et al. [**?**] employed the Lévy Flight algorithm to explore the unknown environment. The dispersion is enhanced by adding the APF technique for producing the repulsion among the robots. However, the robot may fail to escape from the dead zone due to the APF method has a local optimum problem.

Wei et al. [**?**] implemented the inspection strategy by combining the APF and particle swarm optimization (PSO) algorithms to overcome the local optimum problem by optimizing the speed and position of particles.

Wang et al. [**?**] introduced a potential field based on the information gain and path cost, in which the robot can find the optimized trajectory to avoid being trapped in local minima.

Huang et al. [**?**] utilized the APF method to cover the area by the formation control of the multi-robot system. Despite all the research effort, there is still a lack of planning for collision avoidance between multiple robots when simultaneously access to the goal under the potential field.

Following is all the coverage path planning, if you want to know more (e.g. advantages and disadvantages), please read [**?**].

1. Randomized algorithms: Robot swarm model for searching task; unpredictable rapid search.
2. Spanning tree coverage algorithms: Computes a spanning tree and the robot visits each cell once in a configuration space without backtracking search.
3. Artificial potential field algorithm: Simple math model to develop obstacle avoidance strategies; Avoid collision between multiple robots.
4. Sampling-based planning algorithms: handle the CPP problem to find the uncovered area especially in large state space.
5. Evolutionary algorithms: Optimally assign a group of robots to execute the number of tasks in cooperation to achieve system goals. One of the example is particle swarm optimization algorithm.

3.3 Question

1. Frontier-based exploration vs. Lévy walk vs. density-weighted coverage control — which suits a *known-geometry, unknown-state* field? Note that our field layout is known in advance; classic frontier exploration assumes unknown geometry, so does it still apply, or does it degrade to coverage control?

Ans: A coverage path planning, in our case the potential field, will be most suitable for the known-geometry, unknown-state field. There is no need of extra exploration algorithms (like frontier-based exploration). It basically degrade to coverage control in this case.

2. How does a robot pick a target using only local knowledge plus received messages? What cost function (distance, region size, staleness, pest probability)?

Ans: The robot can select its next target by evaluating a weighted combination of different utility or cost terms derived from its local observations and the information received from other robots. Additional terms can also be incorporated depending on the objectives and constraints of the swarm, such as the robot's battery level, the current swarm composition.

3. Can our pest-probability map be used directly as the importance density in coverage control? What breaks if the density changes faster than the robots converge?

Ans: As explained above, it can be the input(utility) in the target decision making. The swarm might chasing the target, therefore, the time-scale of density changes need to be monitored.

4. Potential fields: what do we do about local minima, and is the extra complexity of a predictive variant warranted at our scale?

Ans: One example of the research above incorporate the PSO to overcome the local optimum problem. In my opinion, it is worth to implement a predictive variant which might be useful when the crop field is large or the exploring resource is scarce.

5. How do we keep robots from clumping while preserving communication links? Quantify the trade-off.

Ans: We need to prevent the robots from colliding (and clustering), while ensuring there is enough neighbour robots within radio range/line of sight.

6. Structured rows probably beat free-space navigation — is there row-following literature that makes the whole APF question moot?

Ans: I haven't read the related literature yet.

7. How potential fields can achieve path finding/navigation?

Ans: The potential field is defined over the entire free space, and at each time increment, we compute the potential field at the robot position, and then calculate the induced force by this field. The result is that the robot will move according to this force. If the robot's space has an obstacle, we make it generate a repulsive field in the surrounding space, which when the robot approaches the obstacle, pushes the robot away from it. If we want to make the robot going to the goal and avoiding the obstacle, then we superimpose an attractive field around the goal and a repulsive field around each obstacle in the robot's space.

8. What about obstacles like a hole? Should we send some "dangerous area" signal? For another robot it might be an attractor - rescue robot/scavenger robot.

Ans: No related literatures read. I think the head should be able to identify the hole as an obstacles (user requirement from us). It is the design question.

9. How do we model the robots brain. Does it run a decision loop, collecting data from the sensors and other robots at each frame? If so what does the decision loop consist of?

Ans:

10. How does the robot decide on the next objective? It has to aggregate a multitude of decision drivers: it's own capabilities, the field objectives from other robots, the objectives it knows personally, whether it's doing something right now, it's location, it's status like the battery charge. Are all decisions made using the same system or should we introduce some pre-programmed sub-routines like looking for a nearby charging station when battery is low.

Ans:

11. How do we store the location based data (like plant infestation) in the robot's brain? Is it represented by a 2D grid cell or just markers in some continuous space? How do we model the decay of the information, counting separate events (is infestation observed by robot A the same as the one observed by B?), updates from other robots? How do we model navigation in this in-memory map?

Ans:

3.4 Interesting Literature

Following is some literatures that has not been cited but might be useful for other research topic:

1. Introduction to Mobile Robot Control
2. Multi-robot collaboration for robust exploration
3. Multi-Robot Exploration In Network-Uncertain Indoor Environments
4. Auction-based multi-robot routing
5. Multi-robot exploration and rendezvous on graphs
6. Swarm intelligence in cellular robotics system
7. New Advancements in Swarm Algorithms: Operators and Applications
8. Swarm Intelligence: From Natural to Artificial Systems

[11pt,a4paper]article

[T1]fontenc [utf8]inputenc textcomp lmodern [margin=2.5cm]geometry amsmath,amssymb booktabs tabularx enumitem microtype parskip [hidelinks]hyperref

Q4: Task Allocation and Decision Making Obi September 14, 2026

This section answers two questions. Given what a robot currently believes about the field, what should it do next? And once it is doing something, when should it stop and do something else instead?

I have written it on the assumption that stigmergy supports the belief-sharing layer in Task 3 rather than replacing it. Stigmergy means coordinating through marks left in the shared environment instead of by sending messages. Ants do this with pheromone. Our robots would do it with a shared digital map. The last section (§**??**) sets out how Task 3 and Task 4 fit together, so that the two write-ups do not contradict each other.

I assume four kinds of robot throughout:

- **Monitoring** robots. Light and fast. They scout and detect.
- **Killer** robots. They treat pests by spraying, laser or mechanical removal. Heavy, slow, and they carry a consumable that runs out.
- **Scavenger** robots. They remove dead pest matter and culled leaves, which spread disease if left.
- **Battery** robots. They bring energy to the others, either by charging them or by swapping packs.

4 Decision Making

4.1 Gerkey and Matarić (2004), IJRR 23(9):939–954

This paper gives us vocabulary rather than an algorithm. Its main point is that a lot of earlier work on sharing tasks between robots was really solving standard optimisation problems, but without realising it. As a result, people invented rough heuristics in cases where a known optimal algorithm already existed.

The authors sort allocation problems along three axes.

- **ST or MT.** A single-task robot does one job at a time. A multi-task robot does several at once.
- **SR or MR.** A single-robot task needs one robot. A multi-robot task needs a group.
- **IA or TA.** Instantaneous assignment decides only what to do right now, with no model of what might arrive later. Time-extended assignment gives robots a schedule and plans ahead.

A problem is then written as a triple, such as ST-SR-IA. One warning, because I misread this myself at first. "Instantaneous" describes how far ahead the system plans. It says nothing about how quickly the decision is made.

The complexity results are the part we actually need. NP-hard means there is no known way to find the exact best answer quickly once the problem gets large, so we would have to settle for an approximation anyway.

Class	Equivalent problem	How hard
ST-SR-IA	optimal assignment	easy (Hungarian method or auctions)
ST-MR-IA	set partitioning	very hard (NP-hard)
MT-SR-IA	set covering	hard (NP-hard)
ST-SR-TA	scheduling, or vehicle routing with travel costs	hard (NP-hard)

The short version is this. Assigning one robot to each job, right now, is easy. Add groups of robots, or add planning ahead, and the problem becomes hard. Most of Part 5 follows from that one sentence.

Utility is defined as $U_{ij} = Q_{ij} - C_{ij}$. That is the expected quality of the work minus the expected cost of doing it. Utility is set to zero whenever a robot cannot do the job at all. This turns out to be how we handle having four different robot types, and it is simpler than I expected. We do not need a separate system of roles and permissions. We just need a table of utilities that is mostly zeros, because a killer robot scores zero on clearing debris and a battery robot scores zero on scouting. The allocator never has to know that robot types exist. It just picks the highest numbers, and impossible pairings lose automatically because they score zero.

The authors also measure how good existing systems are, using competitive bounds. A competitive bound compares an algorithm against an imaginary perfect planner that knew the whole future in advance. Saying an algorithm is 3-competitive means it never costs more than three times what the perfect planner would have spent, no matter what happens. ALLIANCE is at least 2-competitive, BLE is 2-competitive, and the simple greedy auction in MURDOCH is 3-competitive when tasks arrive one at a time.

That last number is the useful one. The authors also show that no algorithm making decisions in real time can do better than 3. So greedy is not just acceptable here. It is the best available. This lets us say something stronger than "we used a simple rule". We can say the exact problem is too hard to solve, and the simple rule is provably as good as any real-time method can be.

One gap. The paper assumes tasks are independent of each other. Ours are not. I still need to read Korsah, Stentz and Dias (2013), which adds a fourth axis for how tasks depend on one another: NN for none, ID for dependencies inside one robot's own schedule, XD for dependencies across different robots' schedules, and CD for complex cases. We need XD at minimum. See §5.1.

4.2 Font Llenas et al. (2018), ANTS 2018, LNCS 11172:135–149

Up to 100 Kilobots collect food from several source areas and carry it back to a central depot. They coordinate using nothing but virtual pheromone. The robots cannot talk to each other, cannot sense each other, and do not remember where the sources are.

The pheromone comes from ARK, a system where an overhead camera tracks the robots and a shared matrix stores the pheromone field. Cells are 6.7 mm across. Each deposit adds $\varphi = 100$. The field evaporates at $\epsilon = 0.08$ and spreads at $\gamma = 0.01$ every half second. Each robot can only tell whether pheromone is present or absent, in four 45° sectors ahead of it, up to about 3.5 cm away.

Here is the mechanism worth copying. Each source has a quality score v between 0 and 10. On the way back to the depot, a robot deposits pheromone with probability $P = v/v_{\max}$.

Note what this does. The size of each deposit is always the same. What changes is how often the robot deposits at all. A top-quality source produces a solid, unbroken trail. A middling source produces a dotted trail with gaps in it. So quality is stored in how continuous the trail is, not in how strong it is. They had to do it this way because the Kilobots can only detect presence or absence, so a stronger trail would look identical to a weaker one.

The robots have no memory. Their whole behaviour is four states: wander randomly, head to the depot, follow pheromone, turn back. All of the colony's memory lives in the environment instead of in the robots.

Findings that carry over to us:

1. The further away a source is, the less food comes back from it. The swarm sends more robots to sources that are closer and better, even though no robot ever calculates that trade-off.
2. The trade-off between quality and distance is measurable. With one source of quality 10 at 1 m and another of quality 5, the swarm only switches to the poorer source once it is more than 0.2 m closer. Nobody designed that exchange rate. It falls out of the deposit rule.
3. Swarm size changes the behaviour. With 50 robots the swarm is genuinely picky and moves to the better source. With 200 robots there are enough of them to cover every path at once, so the swarm effectively stops choosing. **Too many robots destroys the selectivity we would be relying on.** This is the result to show anyone who argues for a bigger swarm.
4. Crowding puts a ceiling on performance. Total output still rises as you add robots, but the output per robot keeps falling because they get in each other's way. There is an ideal density.
5. The authors suggest two extensions that I think we should take. One is adjusting how much pheromone gets dropped and how often, so the quality and distance balance can be tuned deliberately. The other is adding a repellent pheromone, which pushes robots away rather than pulling them in.

One caveat, which I would rather we raise ourselves than have a reviewer raise for us. **ARK is a central system.** The robots are decentralised, but the pheromone itself lives in one shared matrix on a server. That is exactly the assumption PheroCom criticises. So we should cite this paper for its result about quality-weighted deposits, and not as evidence that decentralised pheromone works in a real field.

4.3 Tinoco and Oliveira (2022), PheroCom, arXiv:2202.13456

This paper targets the weakness above. Earlier virtual-pheromone work either used beacon robots that sit still and do no useful work, or floors fitted with RFID tags, or a central agent holding the pheromone field. None of these would work across a real farm.

In PheroCom, each robot keeps **its own pheromone map in its own memory**. That map changes in three ways. The robot adds its own deposits. The map evaporates over time. And the robot merges in pieces of map from other robots it meets nearby. Sharing is modelled on the way ants signal through vibration, and works like gossip: robots pass information along when they happen to meet, with no schedule and no central authority. The authors test this in their own simulator and in Webots, across different environment shapes, sizes and swarm sizes. I have only skimmed the Webots experiments so far. The model description in sections 3 and 4 is the part that matters for us.

What this gives us:

- No dependence on fixed infrastructure. The pheromone field exists only in the robots, so losing the base station does not destroy it.
- No waiting. A robot acts on whatever map it currently has.
- Graceful failure. Every robot's map is a bit incomplete and a bit out of date by design. For twenty hectares with patchy radio, that is the right trade.

Weaknesses, which I think we should state openly:

- Maps drift apart. Two robots can believe different things about the same cell, and the paper offers no guarantee that they converge. It only shows that this works well enough in practice.
- Double counting. A single number in a cell cannot tell you whether one robot saw something clearly or three robots half-saw it. Worse, when maps get merged and re-shared, the same original sighting can be counted again and again. For us this matters, because that number would be driving a spray decision that cannot be undone.
- Shared coordinates. Robots have to agree on where cells are before they can merge maps, and the paper treats this as its central difficulty. See §6.6. I think crop rows solve this for us almost for free, which is the nicest thing I found while writing this.
- Useful follow-up work: Dimitrov and Hristov (2024) on how well incomplete pheromone maps hold up, and the ANTS 2024 work on what happens when trails are misleading.

4.4 How the three fit together

Gerkey and Matarić tell us what kind of problem we have and how hard it is. Font Llenas et al. show that a swarm using only pheromone can still balance how bad a problem is against how far away it is, and that this balance can be tuned. PheroCom shows how to hold the pheromone field without a central server, and is the right paper to cite against any work that quietly assumes one.

Together they support the combination we want. Each robot holds its own decaying map and shares it opportunistically, following PheroCom. Deposits are weighted by how severe the problem is, following Font Llenas. Those values then feed the benefit and cost terms of a simple greedy assignment whose worst case is already known, following Gerkey and Matarić.

5 The questions

5.1 What kind of allocation problem do we actually have?

My answer: **XD [ST-SR-TA], with an ST-MR-IA problem sitting inside it, implemented as ST-SR-IA repeated over and over.**

Taking that apart.

Single-task robots (ST). A killer robot that is spraying is not also scouting, and a battery robot in the middle of a transfer is committed. There is one exception, which I deal with in §5.7. Robots do sense things while driving, which arguably counts as a second job. If we treat that sensing as a free side effect rather than an assigned task, the classification holds.

Mostly single-robot tasks (SR), with exceptions. Most treatments need one robot. But a large infestation might need several killers working together to stop it spreading. And **every battery rendezvous needs two robots by definition**, since both have to be in the same place at the same time. That part is ST-MR-IA, which is the NP-hard set partitioning problem.

Time-extended in principle, instantaneous in practice. The future genuinely matters to us. Batteries run down, pest populations grow, and harvest is a fixed deadline. But I would still implement it as a series of right-now decisions with only a short look ahead. Planning far ahead is NP-hard, and in any case our information goes stale faster than a long plan would survive.

Cross-schedule dependencies (XD), not just in-schedule (ID). I do not think we can avoid this. The battery robot's schedule limits what every other robot can do. Scavenger jobs only exist because killer robots created them. And two killers sent to the same patch are worth less than twice one killer, because the second one adds little. ID would only capture dependencies within a single robot's own plan, which misses all three of these.

Why this classification is worth the space. It lets us replace "we chose a simple swarm rule" with "the exact problem is hard in three separate ways, and the simple rule is provably the best any real-time method can do". That second claim is Gerkey and Matarić's, not ours, which is exactly why it is worth using.

What I would build: each robot keeps a local list of nearby jobs and claims the best one it can reach, using leases as described in §5.4. This works like MURDOCH, except that nobody runs the auction. The shared pheromone field takes the auctioneer's place.

5.2 The utility function: severity, travel and staleness

I would rather not tune weights by hand across three things measured in different units. Instead, convert all of them into money, specifically the crop value we expect to save. Then divide by robot-time, which turns the result into a rate.

Starting from $U = Q - C$ and expanding:

$$U(r,t) \;=\; \frac{V_{\text{crop}}(x)\; s\; c\; g(\Delta_{\text{harvest}})\; \eta_r \;-\; \left(\kappa_e\, E_r(d) + \kappa_c\, m_r\right)}{d/v_r \;+\; \tau_{\text{service}}} \tag{2}$$

lll

Term	What it is	Where it comes from
$V_{\text{crop}}(x)$	crop value per m^2 at that spot (€/m^2)	farm data, fixed
s	how bad the infestation is, 0 to 1	monitoring belief
c	how much we trust that estimate, given its age	explained below
$g(\Delta)$	how fast the problem is growing	pest population model
η_r	how well this robot can treat it, 0 to 1	0 if it cannot at all
κ_e, κ_c	cost of energy and of consumable	procurement
d/v_r	travel time: distance divided by this robot's speed	
τ_{service}	how long the job itself takes	

Why divide by time rather than just subtracting cost? Because then the balance between how valuable a job is and how far away it is falls out on its own, instead of being set by a weight we invented. A job worth twice as much but three times further away loses, automatically. This is also the standard framing from foraging theory, which Font Llenas et al. point to in their discussion as the principled version of their own rule. And it puts a fast monitoring robot and a slow killer robot on the same scale, which we need if they are to compare jobs sensibly.

Staleness works in opposite directions for different robots. This took me a while to see and I think it is the least obvious part.

For a treatment job, old information makes the job less attractive. The pest may have moved, or someone may have dealt with it already, so the trip could be wasted. For a monitoring job, old information makes the job *more* attractive. Not having looked somewhere recently is exactly what makes it worth looking there.

So the same age value gets read two ways:

- killers and scavengers multiply the benefit by $c = e^{-\lambda \Delta t}$, which shrinks as the information ages;
- monitors use $1 - e^{-\rho(x)\Delta t}$, which grows with age. This is the chance that something new has appeared since the last visit. See §5.6.

One field, two readings. It is also the simplest way to explain why the same marks attract treatment robots and push away scouts.

Once everything is in the same units we are left with three things to tune: λ, how fast we lose faith in old sightings; $\rho(x)$, how likely a problem is to appear in a given place; and how cautious we want to be about s. Three parameters is a number we can justify tuning by experiment. Seven hand-picked weights is not.

Feasibility is a gate, not part of the score. If a job would cost more energy than the robot has left, counting the energy it needs to get back to a charger, then its utility is zero. Not reduced, zero. We must never let a high enough severity outweigh a hard limit, because a killer robot stranded in row 40 is worse than any untreated patch. The same applies to the legal limits in §6.5. Too much wind, a re-entry period that has not expired, or a no-spray buffer all set the score to zero rather than lowering it.

5.3 When should a robot abandon what it is doing?

Not every action is equally easy to interrupt, and I think treating them all the same is what causes trouble. Either the swarm keeps changing its mind, or it ignores urgent news. Three tiers:

1. **Cannot be interrupted.** A spray pass already underway, a docking handshake, a laser firing. Stopping halfway wastes the consumable and leaves a partly treated patch, which can help pests build resistance. We should design these actions to last seconds rather than minutes, so this tier never blocks a response for long.
2. **Can be interrupted at checkpoints.** Treating a large patch in several passes. Reconsider between passes, not during one.
3. **Can be interrupted freely.** Driving. Nearly all reassignment should happen here. It costs almost nothing, because the distance already travelled is lost either way.

Some things override all of the above without any comparison: critically low battery, a safety stop, a human instruction, or a hardware fault. These should not be written as utility comparisons at all.

Thrashing means a robot switching between jobs repeatedly without finishing any of them. Hysteresis is the general name for rules that make switching harder than staying. I would use all five of the following, because each one blocks a different cause.

- **Require a clear margin.** Only switch from job t to job t' if $U(t') > (1 + \varepsilon)\, U(t) + C_{\text{switch}}$. Set ε around 0.2 to 0.3, and let C_{switch} cover driving back and setting up again. This stops switching caused by small fluctuations.
- **Minimum time before reconsidering, $\tau_{\min}$.** A robot simply cannot change its mind more often than this. It is the same idea as a refractory period in a nerve cell, or backoff in TCP. I think this is the single most effective fix and it costs one timer.
- **A bonus for progress already made.** Add $\beta \cdot \text{progress}$ to the current job's score. Strictly this is irrational, because money already spent should not affect what you do next. But it is deliberately irrational, and it is roughly what ants do when they stay loyal to a site. I would rather describe it honestly as trading a little optimality for stability than pretend it is optimal.
- **Smooth the beliefs before using them.** Allocate using a running average of severity, not the raw value. Information arriving ten times a second must not cause decisions ten times a second.
- **Randomise tie-breaks and stagger decision times.** Otherwise a single update sent to everyone at once makes the whole swarm switch simultaneously. That is the classic swarm oscillation, and staggering the clocks prevents it.

One more: use different thresholds for starting and stopping. The severity that makes a robot commit to a patch should be higher than the severity that makes it leave. With a single threshold, a patch sitting right at that value will cause endless flip-flopping.

Worth measuring: **number of job switches per job completed**. If it goes above roughly 0.5, our settings are wrong. No other metric will catch this.

5.4 How do we stop five robots converging on one infestation?

Use explicit claiming for treatment and implicit repulsion for scouting. Both, because they fail in different ways.

The key point is that the two cases are not symmetric. **Sending too many robots is expensive for treatment and harmless for monitoring.** Five killers on one patch means wasted chemical, robots blocking each other (Font Llenas et

al. measured this directly), and four other patches ignored. Five monitors with overlapping routes is only mildly wasteful. Using one mechanism for both is the mistake.

Explicit claiming, for treatment. A claim is a short message: `(task_id, robot_id, ETA, severity_estimate, lease_expiry)`.

The important part is that **a claim is a lease, not a lock**. It expires unless the robot renews it. So if a robot breaks down, runs flat, or drives out of radio range, its claim simply times out and the job becomes available again. Nobody has to detect the failure, and nothing needs a coordinator to clean up.

The semaphore idea I suggested earlier is the natural extension of this, and I have written it up that way. A job advertises how many robots it needs, say three killers, and claims fill those slots. This is the same group-assignment structure as ST-MR-IA from §5.1, but made manageable by fixing the group size in advance rather than searching for the best grouping. Two changes I would now make to the original idea:

- **Slot counts should decay**, as in the original note. An out-of-date slot count is worse than none at all. Decay it on the same clock as the pheromone, so we only have one notion of freshness rather than two.
- **Slots should be soft rather than hard.** A full slot count should make a robot less likely to join, not forbid it. Suppose all three slots are taken, but a robot is 2 m away with a full tank while the three claimants are 200 m out. It should still be able to take the job. Hard slots combined with out-of-date information produces jobs that nobody is allowed to do, which is the worst outcome available.

Implicit repulsion, for scouting and as a backup. Robots leave a short-lived repellent mark where they are and along the route they intend to take. Others are pushed away from it. This needs no addresses, no identities and no replies, and it works with the radio switched off entirely. It fails at exactly the same time as the claiming layer, which is the point of having it. Font Llenas et al. suggest repellent pheromone as the missing ingredient in their own system, and Dimitrov and Hristov found that merging maps plus repulsion beat random search on every coverage measure, at every swarm size.

What links the two layers: a robot becomes less likely to join a job the more robots it thinks are already on it. It works out that number from claims when it has them, and from repellent concentration when it does not. One rule, two sources of information, and performance that fades gradually rather than switching between two different modes.

5.5 Could pheromone replace the belief-sharing layer completely?

No. But I want to be specific about what we would lose rather than just asserting it. Six things, roughly in order of how much they cost us.

1. **It cannot carry meaning.** A single number in a cell cannot say "*Tuta absoluta*, severity 0.7, on leaf undersides, row 12, seen by robot 4 at 09:41 in good light". You can run a few separate pheromone types, as ants do, but not many, and they still cannot carry structured detail. Our killer robots need to know *which* treatment to use, and a gradient cannot answer that.
2. **It cannot count evidence or track its source.** One strong sighting and three weak ones look identical once they are added into the same number. And because robots merge and re-share maps, one original sighting can inflate that number repeatedly as it circulates. Since spraying cannot be undone and costs money, we need to distinguish confirmed from rumoured.
3. **It cannot be corrected.** Pheromone fades on a timer, not in response to new evidence. If a monitor revisits a reported patch and finds it clean, there is no way to say so. We just have to wait for the mark to fade, and in the meantime robots keep being drawn to something that is not there.
4. **It cannot support planning.** Following a gradient answers "which way now". It does not answer "which of these three jobs suits me best given where I am and how much charge I have". It can also get stuck in local dead ends. None of our energy planning can be done this way.
5. **It leaves no record.** Farm regulations require a treatment log: what was applied, where, how much, when, and by which machine. A system that sends no messages cannot produce one, or feed a farmer's dashboard. In the EU this is a legal requirement rather than a nice extra, and on its own it rules out pure stigmergy for anything we actually deploy.
6. **Trail-following is largely wasted on us.** The famous benefit of pheromone trails is finding short paths through open space. In a row crop the path is already decided by the rows. There is essentially one way to reach row 12, segment 30. So we get the benefit of knowing *where to go*, and almost none of the benefit of *how to get there*. Worth saying plainly, because it changes which parts of the 2018 paper actually apply to us.

The split I would propose:

The two fail independently, and the stigmergic layer keeps the swarm doing something sensible while the messaging layer is down. That is the argument for combining them. Stigmergy is not a cheaper version of belief-sharing. It is what makes belief-sharing's failures survivable.

lLLL

Layer	Carries	How	How it fails
Stigmergy (this section)	*where* to go: attention, coverage, recency, avoidance	each robot's own decaying map, merged when robots meet	fades gradually, gets vague
Belief-sharing (Task 3)	*what* is there: pest type, severity, confidence, source, treatment record	structured messages, sent when possible	stops completely

5.6 Is "recently checked" enough on its own for deciding where to look next?

It is cheap and we should use it, but it is not enough by itself. Marking where we have been produces an even sweep of the whole field, and an even sweep is the wrong plan for three reasons.

Risk is not the same everywhere. Pests appear more often in some places than others: at field margins and near hedgerows where they come in from, in damp or shaded low spots, in last season's problem areas, along irrigation lines, at variety boundaries and headlands. The priority for revisiting somewhere should be the chance that something is there and we have not seen it, roughly $1 - e^{-\rho(x)\Delta t}$, where ρ **depends on the location**. Treating ρ as the same everywhere is exactly what a plain recency map assumes, and between a field margin and the middle of the field it is wrong by a large factor.

"Checked" is not yes or no. Driving past at 2 m/s with a wide-angle camera is not the same as stopping and inspecting closely. So each mark needs two numbers, not one: when we looked, and how well we looked. A quick pass should leave a weaker mark, or one that fades faster.

What the robot can physically see. A robot in one row sees that row well and the neighbouring rows poorly. And because leaves block the view, the undersides are never inspected by driving past, however recently we covered the area. Covering the ground is not the same as inspecting the plants. I would record which sensor did the looking alongside the row and segment, or we will keep believing we have checked things we have not.

The extension I would like us to adopt keeps everything stigmergic. **Let the fade rate vary by location**, so that $\epsilon(x)$ is proportional to $\rho(x)$. High-risk places forget that they were checked more quickly, so they get revisited sooner, with no extra machinery. It costs one fixed number per cell. No new messages, no new behaviour, no reasoning. The rule the robots already follow, go where the "checked" mark is weakest, now produces risk-weighted scheduling on its own. We can set $\rho(x)$ initially from what agronomists expect and refine it over the season from where we actually find things. I think this is the one fairly original idea in my section and I would like to keep it.

5.7 Should robots specialise?

Different hardware for different classes, which we already have. But within a class, specialisation should be a tendency that shifts over time rather than a fixed role.

Making every robot identical is not really an option. A spray tank, a battery pack and a high-resolution camera rig are different payloads with different weights, speeds and costs. So the real questions are whether roles within a class should be fixed, and whether any class should do two jobs.

Against fixed roles: they break badly when robots fail, since losing our two scouts would leave us blind. They balance load poorly, because the right ratio of scouts to treaters changes across the season and even during a single day. And somebody has to choose that ratio in advance without knowing what will happen.

The alternative is the response threshold model (Bonabeau, Theraulaz and Deneubourg, and Ferrante et al. 2015 on robot swarms). Each robot carries a threshold for each kind of job, and it takes the job when the demand it senses rises above its threshold. Thresholds drop as a robot does a job more often and rise while it sits unused. So robots drift towards specialising without anyone designing it, and, importantly, they drift back again when the make-up of the swarm changes. That is what makes this robust to losing robots.

The semaphore idea is the explicit version of the same thing, and the two combine neatly. Treat slot counts as targets that shift thresholds rather than as fixed quotas. If the swarm wants three scouts and thinks it has one, every robot's threshold for scouting drops a little. Then decay gives us the right failure mode. When slot information goes stale, we fall back to plain threshold behaviour rather than to an actively wrong allocation. That is a genuinely good property, and I would rather present it as the reason for the design than leave it buried in the implementation.

One point across classes. Killer and scavenger robots should carry cheap cameras. They are driving past plants anyway, the extra cost of a camera and a detection model is small, and it multiplies our monitoring coverage several times over. Strictly this makes them multi-task robots and breaks the ST classification from §5.1. I would handle it by treating that sensing as a free side effect that writes into the shared map but is never assigned as a job, and I would mention the inconsistency openly, because a reviewer will notice it.

Against putting monitoring and treatment on the same machine: spray drift dirties the camera, treatment units are heavy and slow so they scout badly, and a treatment robot covered in chemical driving the scouting routes risks contaminating the crop.

6 Further questions I think we need to answer

These were not on the original list but came up while writing, and several of them affect the answers above.

6.1 Who decides where the battery robot goes?

This is the hardest allocation question we have and it is not on our list. The battery robot is the reason our tasks depend on each other across robots, and if it is scheduled badly then everyone else's availability collapses with it.

What I would do: robots request energy, and the battery robot decides the order. A robot's request should get more urgent **faster than linearly** as its charge falls, so that nearly-flat robots clearly outrank merely low ones and the ordering is never ambiguous. The battery robot then plans a short route through the pending requests, respecting their deadlines, and parks near the middle of expected future demand when it has nothing to do.

Purely reactive scheduling leaves robots stranded. Purely predictive scheduling wastes the battery robot's time driving to robots that did not need it. Everyone else keeps enough charge in reserve to reach a charger unaided (§5.2), so that if the battery robot fails we get slower service rather than lost robots.

One thing we should settle early, because it affects every platform's mechanical design: does the battery robot swap packs or charge in place? Swapping is much faster but forces a common pack design across the fleet.

6.2 Where do scavenger jobs come from?

They are created by killer robots finishing their work, so one robot's output is another's input. Stigmergy suits this unusually well. The killer leaves a debris mark at the treated spot as a side effect of finishing, and scavengers follow those marks. Nobody creates a job, sends a message, or coordinates anything.

I think this is the clearest case in our whole system where the stigmergic layer is genuinely *better* than explicit allocation, rather than just a cheaper backup, and it is worth pointing out for that reason.

6.3 What stops one false alarm pulling in the whole swarm?

The positive feedback that makes stigmergy work is also its characteristic way of failing. Spraying cannot be undone, costs money, and encourages resistance, so a false alarm that spreads is expensive three times over.

What I would require: before any action that cannot be undone, demand confirmation. Either several independent sightings from different robots, which means we need to know who reported what, bringing us back to §5.5, or one sighting the detector is very confident about. Looking and confirming are cheap and reversible. Treating is not. The thresholds should reflect that difference. The ANTS 2024 work on misleading trails covers the general shape of this problem.

6.4 What happens when robots fail?

We have not written this down anywhere, and everything above depends on it. Specifically: how do we notice a robot has died (its lease expires, §5.4), what happens to the job it had claimed (it returns to the pool automatically), what happens to its map (it is lost, which is fine because other robots hold overlapping copies), and how few robots can we drop to before we stop and call a human. We should also set a target, something like still meeting our detection-time target with 30% of the fleet out of action.

6.5 The rules that cannot be enforced locally

Some rules are local and easy to apply to a single decision: wind above the label limit, a re-entry period that has not elapsed, a buffer strip beside a watercourse, a no-spray boundary. Each robot can check these for itself.

Others are totals across the whole season and the whole field: a maximum number of applications, a maximum amount of active ingredient per hectare, required rotation between chemicals to manage resistance. **No system based only on local, fading marks can enforce a total.** This is a real and citable limit on pure stigmergy for our application, and one of the better arguments for combining the two layers, so we should use it.

What I would suggest: hand out the season's allowance as tokens. A robot must hold a token to spray, and robots pass spare tokens to each other when they meet. This is a semaphore over a *resource* rather than over a *role*, so it extends the idea we already have. It also fails in the right direction. If the swarm gets split up, robots run out of local tokens and stop spraying, which is the safe outcome.

6.6 Localisation, and why crop rows help us

PheroCom treats agreeing on coordinates as its central difficulty. Our environment removes most of that difficulty. Crop rows are a strong, permanent structure that a robot can recognise, and RTK-GNSS is already standard equipment on farm machinery.

So I would label the pheromone map by `(row_id, segment_index)` rather than by metric position. This gives us three things:

- The map becomes a list per row instead of a 2D grid. A 500 m row split into 1 m segments is 500 cells, and 100 rows

is 50,000 cells. At a few bytes each across two channels, **every robot can hold the entire field's map in well under a megabyte**. PheroCom's whole argument for incomplete maps largely stops applying to us.
- Merging maps becomes trivial. No alignment, no correcting for drift. Both robots refer to the same cell by the same name, so we just take the fresher or stronger value.
- It survives GNSS dropout, because a robot can work out which row and segment it is in by looking at the rows themselves.

This is probably the strongest domain-specific point I have. The layout of the crop makes decentralised stigmergy much easier for us than it is in the open-environment literature, and I would rather say that explicitly than inherit assumptions from papers written for a different setting.

6.7 What are we going to measure?

We need this agreed before we run any experiment. What I propose:

- **Detection time**: from an infestation starting to us first noticing it.
- **Treatment time**: from noticing it to treating it. Report these two separately. They have different causes, and averaging them hides which one is broken.
- **Total severity over the season**: the headline agricultural result.
- **Chemical used per unit of damage prevented**: this is what catches sending too many robots.
- **Robot-hours per hectare per day**: the cost.
- **Job switches per job completed**: the thrashing measure from §5.3.
- **The spread of how stale our coverage is, not the average.** The worst case is what matters. One corner we never visit is worse than mediocre coverage everywhere.

We should also run the three-way comparison: stigmergy only, belief-sharing only, and both. That is the experimental version of our argument for combining them, and we should plan it now rather than add it later.